

Stack ADT Using Arrays

LIFO Principle, Operations and C Implementation

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

What is a Stack?

Definition

A stack is a linear data structure in which insertion and deletion are performed only at one end, called the **top**.

LIFO: Last In, First Out

If the insertion order is

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40,$

then the removal order is

$40 \rightarrow 30 \rightarrow 20 \rightarrow 10.$

Real-Life Analogy

- Plates are added at the top.
- Plates are removed from the top.
- The last plate placed is removed first.



Stack as an Abstract Data Type

An ADT specifies **what operations are available**, not how they are internally implemented.

Stack Interface

- push(value)
- pop()
- peek()
- isEmpty()
- isFull()

Possible Implementations

- Array
- Linked list
- Dynamic array

Array Representation

A stack of capacity MAX can be declared as:

```
int stack[MAX];
```

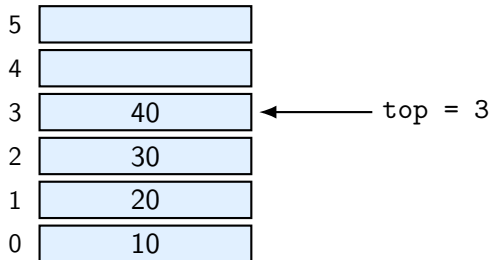
The variable top stores the index of the current top element:

```
int top = -1;
```

- Empty stack: $\text{top} = -1$
- One element: $\text{top} = 0$
- n elements: $\text{top} = n - 1$
- Full stack: $\text{top} = \text{MAX} - 1$

Visual Representation

Assume $MAX = 6$ and the stack contains 10, 20, 30 and 40.



$$\text{Number of elements} = \text{top} + 1 = 4$$

Stack Invariant

For a correctly maintained array stack:

$$-1 \leq \text{top} \leq \text{MAX} - 1$$

Valid stack elements occupy:

`stack[0]` through `stack[top]`

Unused array positions begin at:

`stack[top+1]`

Important

Only the element at `stack[top]` is directly inserted or removed.

Push Operation

Push inserts an element at the top.

- 1 Check whether the stack is full.
- 2 Increment top.
- 3 Store the value at `stack[top]`.

$$\text{top} = \text{top} + 1$$
$$\text{stack}[\text{top}] = \text{value}$$

Time complexity = $O(1)$

Stack Overflow

Overflow occurs when a push is attempted on a full stack.

$$\text{top} == \text{MAX} - 1$$

For $\text{MAX} = 5$, valid indices are:

0, 1, 2, 3, 4

If $\text{top} = 4$, no additional element can be inserted.

Push Function in C

```
1 bool push(int value)
2 {
3     if (top == MAX - 1)
4     {
5         printf("Stack overflow\n");
6         return false;
7     }
8
9     top++;
10    stack[top] = value;
11    return true;
12 }
```

Pop Operation

Pop removes and returns the topmost element.

- 1 Check whether the stack is empty.
- 2 Save `stack[top]`.
- 3 Decrement `top`.
- 4 Return the saved value.

```
value = stack[top]
```

```
top = top - 1
```

Time complexity = $O(1)$

Stack Underflow

Underflow occurs when pop or peek is attempted on an empty stack.

top == -1

There is no valid element at `stack[top]`, so the operation must stop.

Important

Always test the empty condition before reading `stack[top]`.

Pop Function in C

```
1 bool pop(int *removedValue)
2 {
3     if (top == -1)
4     {
5         printf("Stack underflow\n");
6         return false;
7     }
8
9     *removedValue = stack[top];
10    top--;
11    return true;
12 }
```

Logical Deletion in Pop

Suppose `stack[3] = 40` and `top = 3`.

After pop:

`top = 2`

The value 40 may still physically remain in array position 3, but it is no longer part of the logical stack.

Reason

Only positions from `stack[0]` through `stack[top]` are considered valid stack elements.

Peek Operation

Peek returns the topmost value without removing it.

```
return stack[top]
```

Peek does not change:

- top;
- the number of elements; or
- any stack value.

Time complexity = $O(1)$

Peek Function in C

```
1 bool peek(int *topValue)
2 {
3     if (top == -1)
4     {
5         printf("Stack is empty\n");
6         return false;
7     }
8
9     *topValue = stack[top];
10    return true;
11 }
```

Display Operation

To display the stack from top to bottom:

```
for (i = top; i >= 0; i-)
```

For the array order:

10, 20, 30, 40

the stack view is:

40, 30, 20, 10

Display complexity = $O(n)$

Display Function in C

```
1 void display(void)
2 {
3     if (top == -1)
4     {
5         printf("Stack is empty\n");
6         return;
7     }
8
9     printf("TOP\n");
10    for (int i = top; i >= 0; i--)
11        printf("| %d |\n", stack[i]);
12 }
```

Dry Run

Assume $MAX = 5$ and initially $top = -1$.

Step	Operation	Top	Stack from bottom to top
0	Initial	-1	Empty
1	Push 10	0	10
2	Push 20	1	10, 20
3	Push 30	2	10, 20, 30
4	Pop	1	10, 20
5	Push 40	2	10, 20, 40

Complexity Summary

Operation	Time	Reason
Push	$O(1)$	Insert only at top
Pop	$O(1)$	Remove only from top
Peek	$O(1)$	Direct access using top
isEmpty	$O(1)$	One comparison
isFull	$O(1)$	One comparison
Display	$O(n)$	Visits every element

Space used by an array stack of capacity MAX:

$$O(\text{MAX})$$

Complete Program: Definitions

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define MAX 5
5
6 int stack[MAX];
7 int top = -1;
8
9 bool isEmpty(void)
10 {
11     return top == -1;
12 }
13
14 bool isFull(void)
15 {
16     return top == MAX - 1;
17 }
18
19 int size(void)
20 {
21     return top + 1;
22 }
```

Complete Program: Operations

```
1 bool push(int value)
2 {
3     if (isFull())
4         return false;
5
6     stack[++top] = value;
7     return true;
8 }
9
10 bool pop(int *value)
11 {
12     if (isEmpty())
13         return false;
14
15     *value = stack[top--];
16     return true;
17 }
18
19 bool peek(int *value)
20 {
21     if (isEmpty())
22         return false;
```

Complete Program: Display and Menu

```
1 void display(void)
2 {
3     if (isEmpty())
4     {
5         printf("Stack is empty\n");
6         return;
7     }
8
9     for (int i = top; i >= 0; i--)
10         printf("%d\n", stack[i]);
11 }
12
13 int main(void)
14 {
15     int choice, value;
16
17     do
18     {
19         printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Size 6.Exit\n");
20         printf("Enter choice: ");
21
22         if (scanf("%d", &choice) != 1)
```

Complete Program: Switch Cases

```
1  switch (choice)
2  {
3  case 1:
4      printf("Enter value: ");
5      scanf("%d", &value);
6      if (!push(value))
7          printf("Stack overflow\n");
8      break;
9
10 case 2:
11     if (pop(&value))
12         printf("Removed: %d\n", value);
13     else
14         printf("Stack underflow\n");
15     break;
16
17 case 3:
18     if (peek(&value))
19         printf("Top value: %d\n", value);
20     else
21         printf("Stack is empty\n");
22     break;
```

Applications of Stacks

- Function calls and recursion
- Parentheses matching
- Infix, postfix and prefix expressions
- Undo and redo operations
- Browser back navigation
- Depth-first search
- Backtracking
- Reversing data

Interactive Question

Assume $MAX = 4$ and initially $top = -1$.

Perform:

PUSH(5), PUSH(8), POP(), PUSH(12), PUSH(20)

Find:

- 1 the final value of top ;
- 2 the topmost element; and
- 3 the stack from bottom to top.

Interactive Question

Assume $MAX = 4$ and initially $top = -1$.

Perform:

PUSH(5), PUSH(8), POP(), PUSH(12), PUSH(20)

Find:

- 1 the final value of top ;
- 2 the topmost element; and
- 3 the stack from bottom to top.

$top = 2, \quad top\ element = 20$

5, 12, 20

Common Programming Errors

- Incrementing `top` before checking overflow
- Reading `stack[top]` before checking underflow
- Decrementing `top` before saving the popped value
- Using `top == MAX` as the full condition
- Forgetting that the empty condition is `top == -1`
- Returning a special value that may also be valid stack data

References

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, Computer Science Press.
- ② Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.
- ③ Reema Thareja, *Data Structures Using C*, Oxford University Press.

Questions?

Thank You